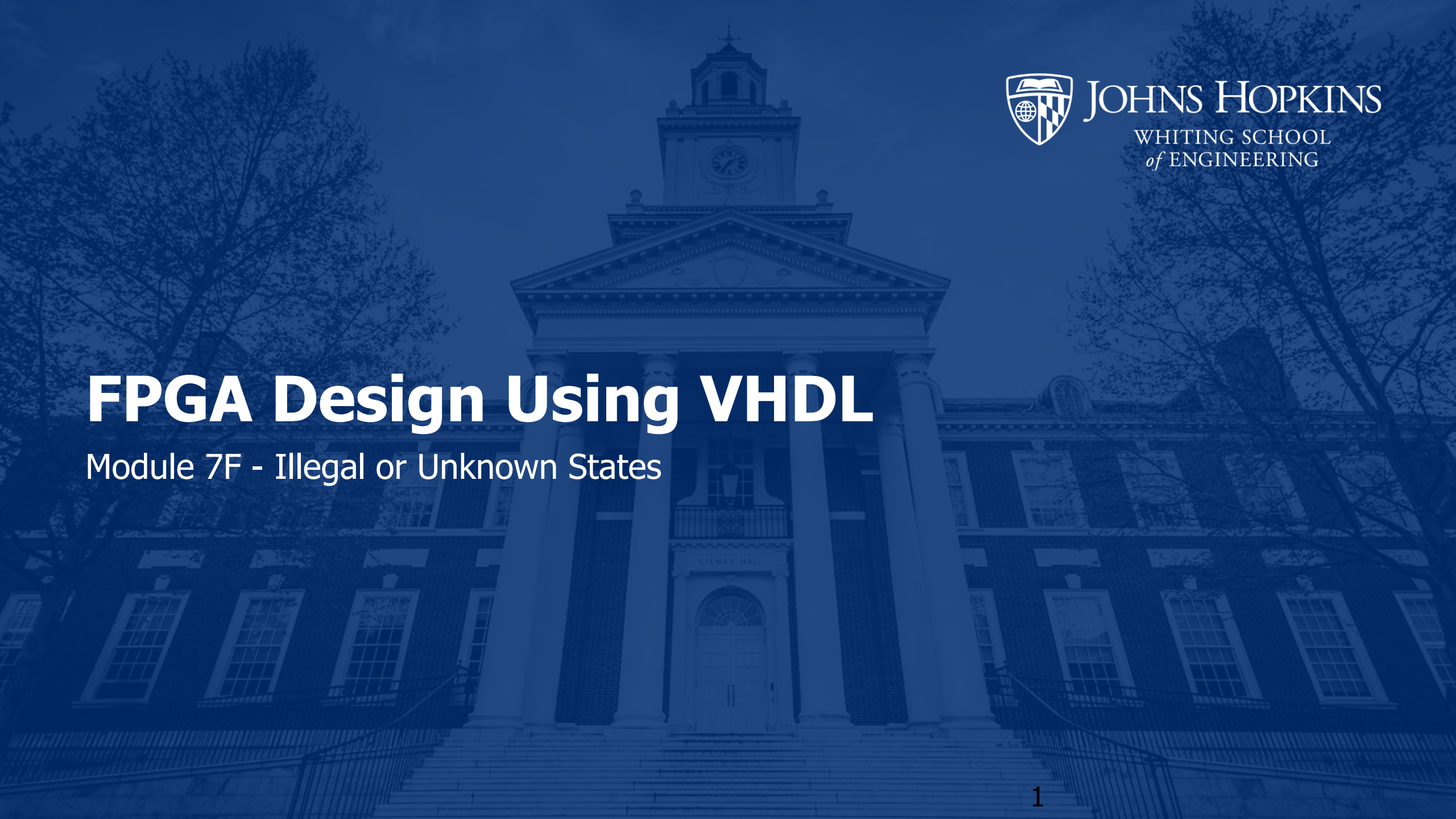


FPGA Design Using VHDL

Module 7F - Illegal or Unknown States

Unknown or Illegal States

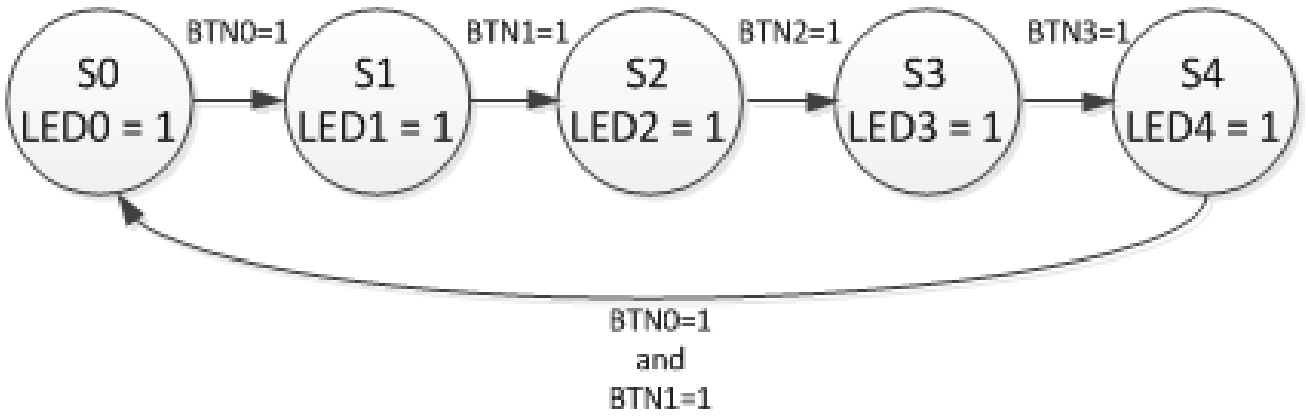
- Given the following states:
  - (idle, init, check, slowchg, fastchg, donechg)
  - 000 001 010 011 100 101
  - If they are encoded as above, we have two illegal states
  - What will design do when those states are encountered?
- Consequences of entering an illegal state is unknown, but frequently result in being "wedged", which you may not recover from

Unknown or Illegal States II

- How can we fall into an illegal state?
  - Faulty Reset Circuitry (or none) could have you power-up in an illegal state
  - Single-Event-Upsets in radiation environments can cause a FF to toggle, leaving your state machine in an illegal state
  - Synchronization errors on inputs
  - Setup/Hold violation
    - Metastability: This will be discussed in detail later in the semester

Unknown or Illegal States III

```
type statetype is (s0,s1,s2,s3,s4);
signal cs : statetype;
```



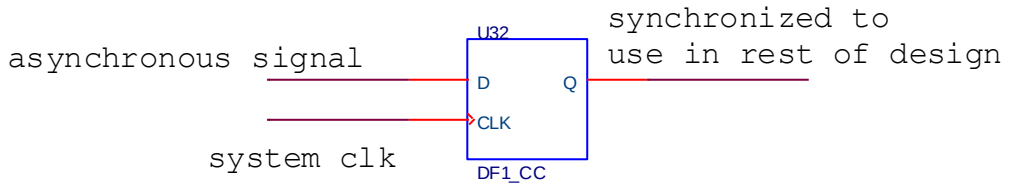
Unknown or Illegal States IV

```
fsm: process(clk50, rst)
begin
   if rst='1' then
      cs <= s0;
   elsif rising_edge(clk50) then
      case cs is
         when s0 => if BTN0 = '1' then cs <= s1; end if; LED0 <='1';
         when s1 => if BTN1 = '1' then cs <= s2; end if; LED1 <='1';
         when s2 => if BTN2 = '1' then cs <= s3; end if; LED2 <='1';
         when s3 => if BTN3 = '1' then cs <= s4; end if; LED3 <='1';
         when s4 => if (BTN0 = '1' and BTN1 = '1') then cs<= s0;
                    end if; LED4 <='1';
         when others => cs <= s0;
      end case;
   end if;
end process fsm;
```

This does NOT fix the problem of getting into an illegal state

Unknown or Illegal States V

- These problems are not specific to Onehot encoding
  - They are simply magnified with the scheme, since there are far more illegal states!
- How do you avoid falling into an illegal state and how to recover
  - Have a reset state
  - Carefully synchronize all inputs



  - Designer can generate reset logic that resets the

```
reset_machine <= '0' when (state = a) or
(state = b)… else '1';
```

  - Make a "safe" state machine

“Safe” State Machines

- Synthesis tools generate additional logic to recover from illegal state
- Recovery state is reset state or power-up state
- Two methods for invoking “safe option”
    1. Via VHDL with attributes
    2. Via XDC (FSM_SAFE_STATE)

"Safe" State Machines II

- The following is a copy/paste from the Vivado UG912 for how safe states can be enabled in RTL:

```
type count_state is (zero, one, two, three, four, five, six, seven);
signal my_state : count_state;
attribute fsm_safe_state : string;
attribute fsm_safe_state of my_state : signal is "power_on_state";
```

- This defines the "power_on_state" which is detected by Vivado. Other options include "reset_state", "default_state", or "auto_safe_state"
  - Definitions of the above can be found in UG912 on FSM_SAFE_STATE